Advanced Algorithms & Network Flows

Complete Handwritten Lecture Notes & Solved Problems

Part III: Advanced Algorithms & Network Flows

Dynamic programming, longest common subsequence, matrix chain multiplication, network flow, bipartite matching, NP-completeness, and SAT problem reductions.

PAGE 1

- **Asymptotics**
- Best-case complexity
- Average complexity
- Worst-case complexity
- Relationship between polynomial, exponential, logarithmic time
- Big-Oh notation: $O(n^d)$, $O(n)$, $O(\log n)$, $O(1)$
- Stable matching problem: (no men and women prefer to be with each other than assigned partner)
- Gale-Shapley algo: $O(n^2)$. Propose and reject.
- Man-optimal.

Graphs (adjacency matrix)

Key Concepts:

- Relationship between degree and number of edges
- Cycles, trees
- Graph Search (DFS & BFS)
- Algorithms for coloring
- Negative Edge Weights in Directed Graphs

Notation:

- $O(m) \leq m \leq n(n-1)/2 \leq O(n^2)$
- $m = n \leq n(n-1)/2 \leq O(n^2)$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$

- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$

- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$

- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$

- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$

- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$
- $m \leq n$
- $m \leq n^2$

- $m \leq n$
- $m \leq n^2$
- \(m

PAGE 3

Greedy Algorithms

1. Greedy stays ahead

- **Interval Scheduling**
- Sort by their finishing times
- Optimal:

$$O(n \log n)$$

2. Structural

- **Approximation algorithms for Vertex Cover, Set Cover (contact every point)**
- **Minimum Spanning Tree Algorithms, and Cycle and Cut Properties**
- **Union Find-Data Structure**

$$O(k \log n)$$

3. Exchange arguments

- $$O(m \log n)$$

  with a binary heap.
- Prim: priority queue

$$O(n^2)$$

  with an array
- Kruskal: union-find data structure
- Cut property: MST contains p by cycle property = MST not contain f. Contradiction.
- $$O(m \log n)$$

for Prim's

- $$O(m \log n)$$

  for union-find.

PAGE 4

Divide and Conquer Algorithms

Recurrences (Master Theorem)

- Algorithms for sorting, multiplication, integer multiplication
- Finding closest pairs, matrix multiplication

Quick Sort

- $T(n) = 2T(n/2) + \Theta(n)$

Merge Sort

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^2$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^c \ (n > b)$
- $= a(aT(n/b^2) + c(n/b)) + cn$
- $= a \log n + n^{1.584}$

Unbalanced Division

- $T(n) = n \log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

- $D(n) = \Theta(n)$

2-D Version

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Divide and Conquer

- $T(n) = 2T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^2$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^c \ (n > b)$
- $= a(aT(n/b^2) + c(n/b)) + cn$

- $= a \log n + n^{1.584}$

Unbalanced Division

- $T(n) = n \log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

- $D(n) = \Theta(n)$

2-D Version

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Divide and Conquer

- $T(n) = 2T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^2$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^c$ $(n > b)$

- $= a(aT(n/b^2) + c(n/b)) + cn$
- $= a\log n + n^{1.584}$

Unbalanced Division

- $T(n) = n\log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

- $D(n) = \Theta(n)$

2-D Version

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Divide and Conquer

- $T(n) = 2T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^2$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^c \ (n > b)$
- $= a(aT(n/b^2) + c(n/b)) + cn$
- $= a \log n + n^{1.584}$

Unbalanced Division

- $T(n) = n \log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

- $D(n) = \Theta(n)$

2-D Version

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Divide and Conquer

- $T(n) = 2T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^2$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^{c}\ (n > b)$
- $= a(aT(n/b^{2}) + c(n/b)) + cn$
- $= a\log n + n^{1.584}$

Unbalanced Division

- $T(n) = n\log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

- $D(n) = \Theta(n)$

2-D Version

- $T(n) = T(n/2) + T(n/2) + \Theta(n)$

Divide and Conquer

- $T(n) = 2T(n/2) + \Theta(n)$

Strassen's Algorithm

- $T(n) = 7T(n/2) + \Theta(n)$

Integer Multiplication

- $T(n) = 8T(n/2) + n^{2}$

Matrix Multiplication

- $T(n) = n^3$

Multiplication of Polynomials

- $T(m) = aT(m/b) + cn^c \ (n > b)$
- $= a(aT(n/b^2) + c(n/b)) + cn$
- $= a \log n + n^{1.584}$

Unbalanced Division

- $T(n) = n \log n + n^{1.584}$

Adjacent

- $T(n) = 2T(n/2) + \Theta(n)$

1-D Version

Dynamic Programming

DP: overlapping sub-problems

Design the recurrence using subproblems, then write the program

Algorithms for sequencing related problems, edit distance, knapsack, weighted interval scheduling, and max weight subset of mutually compatible jobs.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

OPT(i,j) = ∑j=0^i d + OPT(i-1,j-1) otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d + OPT(i-1,j-1)$ otherwise.

$OPT(i,j) = \sum_{j=0}^{i} d$

Network Flows

- Directed graph, no parallel edges.
- Formulate as min-cut.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.
- Directed graph, no parallel edges.

PAGE 7

Linear Programming

Weak:

$$c^T x \le y^T A x \le y^T b$$

$$c^T x = y^T A x = y^T b$$

Minimax Theorems

Duality

Strong Duality Theorem

$$\max c^T x$$

$$s.t.\ Ax \le b$$

$$x \ge 0$$

$$\min y^T b$$

$$s.t.\ y^T A x \ge c$$

$$y \ge 0$$

It doesn't matter whether the row-player or the column-player has to commit to a strategy first.

$$\max c^T x$$

$$s.t.\ Ax \le b$$

$$x \ge 0$$

$$\min y^T b$$

$$s.t.\ y^T A x \ge c$$

$$y \ge 0$$

$$B = c^T$$

$$A = -A^T$$

Randomized Algorithms

$\mathrm{cldet}(B) = \sum_{\sigma:\{n\}\to[n]} \prod_{i=1}^{n} A_{\sigma(i),i}$

Linearity of Expectation $E(\sum_{i=1}^{n} X_i) = \sum_{i=1}^{n} E(X_i)$

Algorithms for Min-Cut, Dominating Set, Flows, Testing when a polynomial is 0.

Hot edges that cross from A to B is minimized.

Let $p(x_1, \ldots, x_n)$ be a polynomial of degree d. Let S be a finite set of numbers.

The probability that k edges of the polynomial are never picked is at least

$(1 - \frac{2}{n}) \cdots (1 - \frac{2}{n}) = \frac{2^k}{n(n-1)}$

If $x_1, \ldots, x_n$ are sampled uniformly from S,

Then if $x_1, \ldots, x_n$ are sampled uniformly from S,

$\mathrm{cldet}(B) = \sum_{\sigma:\{n\}\to[n]} \prod_{i=1}^{n} A_{\sigma(i),i}$

Linearity of Expectation $E(\sum_{i=1}^{n} X_i) = \sum_{i=1}^{n} E(X_i)$

Algorithms for Min-Cut, Dominating Set, Flows, Testing when a polynomial is 0.

Hot edges that cross from A to B is minimized.

Let $p(x_1, \ldots, x_n)$ be a polynomial of degree d. Let S be a finite set of numbers.

The probability that k edges of the polynomial are never picked is at least

$(1 - \frac{2}{n}) \cdots (1 - \frac{2}{n}) = \frac{2^k}{n(n-1)}$

If $x_1, \ldots, x_n$ are sampled uniformly from S,

Then if $x_1, \ldots, x_n$ are sampled uniformly from S,

cldet(B) = $\sum_{\sigma:\{n\}\to [n]} \prod_{i=1}^{n} A_{\sigma(i),i}$

Linearity of Expectation $E(\sum_{i=1}^{n} X_i) = \sum_{i=1}^{n} E(X_i)$

Algorithms for Min-Cut, Dominating Set, Flows, Testing when a polynomial is 0.

Hot edges that cross from A to B is minimized.

Let $p(x_1, \ldots, x_n)$ be a polynomial of degree d. Let S be a finite set of numbers.

The probability that k edges of the polynomial are never picked is at least

$(1 - \frac{2}{n}) \cdots (1 - \frac{2}{n}) = \frac{2^k}{n(n-1)}$

If $x_1, \ldots, x_n$ are sampled uniformly from S,

Then if $x_1, \ldots, x_n$ are sampled uniformly from S,

cldet(B) = $\sum_{\sigma:\{n\}\to [n]} \prod_{i=1}^{n} A_{\sigma(i),i}$

Linearity of Expectation $E(\sum_{i=1}^{n} X_i) = \sum_{i=1}^{n} E(X_i)$

Algorithms for Min-Cut, Dominating Set, Flows, Testing when a polynomial is 0.

Hot edges that cross from A to B is minimized.

Let $p(x_1, \ldots, x_n)$ be a polynomial of degree d. Let S be a finite set of numbers.

The probability that k edges of the polynomial are never picked is at least

$(1 - \frac{2}{n}) \cdots (1 - \frac{2}{n}) = \frac{2^k}{n(n-1)}$

If $x_1, \ldots, x_n$ are sampled uniformly from S,

Then if $x_1, \ldots, x_n$ are sampled uniformly from S,

cldet(B) = $\sum_{\sigma:\{n\}\to [n]} \prod_{i=1}^{n} A_{\sigma(i),i}$

Linearity of Expectation $E(\sum_{i=1}^{n} X_i) = \sum_{i=1}^{n} E(X_i)$

Algorithms for Min-Cut, Dominating Set, Flows, Testing when a polynomial is 0.

Hot edges that cross from A to B is minimized.

Let $p(x_1, \ldots, x_n)$ be a polynomial of degree d. Let S be a finite set of numbers.

The probability that k edges of the polynomial are never picked is at least

$$\left(1 - \frac{2}{n}\right) \cdots \left(1 - \frac{2}{n}\right) = \frac{2^k}{n(n-1)}$$

\(\text{If } x_1, \ldots, x_n \text{ are sampled uniformly from } S,\

decision problems:

problems with "yes" or "no" answers

NP: Nondeterministic Polynomial-time

where P() is some poly.

poly-time: P(x) ≤ poly steps

Definition of NP: decision problems for which there exists a poly-time algorithm.

P: P() algo.

NP: P() certifier (checker)

NP-completeness: NP-complete problems for which there exists a poly-time algorithm.

EXP: EXP() algo.

P ⊆ NP ⊆ EXP

NP = EXP?: is the decision problem as easy as the certification problem?

NP-Complete: a problem y is NP-complete

if for every problem X in NP, X ≤ P y

(Problem X polynomial reduces to problem y)

if any instance of problem X can be solved using a polynomial number of standard computational steps, then y is NP-complete.